

Lab 3

Introduction to Addressing Modes

Prof. Charan Bhaskar

Addressing Modes

The term addressing modes refers to the way in which the operand of an instruction is specified. The addressing mode specifies a rule for interpreting or modifying the address field of the instruction before the operand is actually executed.

Advantages of Addressing Modes

1. To give programmers facilities such as Pointers, counters for loop controls, indexing of data and program relocation.
2. To reduce the number bits in the addressing field of the Instruction.

Addressing modes for 8086 instructions are divided into two categories:

- 1) Addressing modes for data
- 2) Addressing modes for branch

The 8086 memory addressing modes provide flexible access to memory, allowing you to easily access variables, arrays, records, pointers, and other complex data types. The key to good assembly language programming is the proper use of memory addressing modes.

An assembly language program instruction consists of two parts:

Opcode and Operand

IMPORTANT TERMS

- **Starting address** of memory segment.
- **Effective address or Offset:** An offset is determined by adding any combination of three address elements: **displacement, base and index**.
 - **Displacement:** It is an immediate value given in the instruction.
 - **Base:** Contents of base register, EBX or EBP.
 - **Index:** Content of index register ESI or EDI.
- **To load base address of memory location:**

MOV EAX, OFFSET sum OR LEA EAX, sum
--

According to different ways of specifying an operand by 8086 microprocessor, different addressing modes are used by 8086.

Some of the addressing modes used by 8086 microprocessor are discussed below:

1. Implied mode:

In implied addressing the operand is specified in the instruction itself. In this mode the data is 8 bits or 16 bits long and data is the part of instruction. Zero address instruction are designed with implied addressing mode.

Example:

CLC (used to reset Carry flag to 0)

2. Immediate Addressing mode:

In this mode data is present in address field of instruction. Designed like one address instruction format.

Note: Limitation in the immediate mode is that the range of constants are restricted by size of address field.

Example:

MOV AL, 35H (move the data 35H into AL register)
--

3. Register mode:

In register addressing the operand is placed in one of 8 bit or 16 bit general purpose registers. The data is in the register that is specified by the instruction.

Here one register reference is required to access the data.

Example:

MOV AX, CX (move the contents of CX register to AX register)
--

4. Register Indirect mode:

In this addressing the operand's offset is placed in any one of the registers BX, BP, SI, DI as specified in the instruction. The effective address of the data is in the base register or an index register that is specified by the instruction.

Here two register reference is required to access the data.

Since masm is using 32 bit addressing mode, the address we specify must be of 32 bit.

The 8086 CPUs **let you access memory indirectly** through a register using the register indirect addressing modes.

Example:

MOV AX, [EBX] (move the contents of memory location addressed by the register EBX to the register AX)

5. Direct addressing/ Absolute addressing Mode (symbol []):

In this addressing mode the 32-bit effective address of the data is the part of the instruction.

Here only one memory reference operation is required to access the data.

MOV CX, [007E4000h] (move the contents of memory location 007E4000h to register CX, you might get access violation exception at runtime since address access allocation varies each time)

6. Indexed addressing mode:

The operand's offset is the sum of the content of an index register ESI or EDI and displacement.

Example:

MOV EAX, [ESI +05]

7. Based Indexed Addressing:

The operand's offset is sum of the content of a base register EBX or EBP and an index register ESI or EDI.

Example:

ADD EAX, [EBX+ESI]

Lab Work

- Implement following program, and explain in detail each line of the code.
- Mention which addressing modes are used and how. (Attach screenshots of the memory window, where memory locations are being used to justify the explanation.)
- For the instructions marked **red**, explain what variable (from memory) it is referencing to store into **BX** and **CX** registers respectively.

```
.386
.model flat,stdcall
.stack 4096
ExitProcess proto,dwExitCode:dword
```

```
.data

sum WORD ?
var1 WORD 5

.code
main proc

    MOV AX, 0FFFFh
    MOV BX, 7h
    ADD AX,BX
    CLC
    MOV sum, AX
    MOV AX, BX

    LEA EBX, var1
    MOV AX, [EBX]

    MOV ESI, OFFSET sum
    MOV BX, [ESI + 02]

    LEA EBX, sum
    MOV ESI, 02
    MOV CX, [EBX + ESI]

    invoke ExitProcess, 0

main endp
end main
```